Pre-Symptomatic Wheat Stem Rust Detection — 10-Day Plan

Oct 19, 2026 · @taha

Project context (give this to Claude Code first)

Goal: detect wheat stem rust infection 4–5 days after inoculation, before symptoms are visible (day 6–7), from hyperspectral images. Then explain which wavelengths make that possible and how few bands a cheap sensor would need.

How to use this plan with Claude Code: save this section as `CLAUDE.md` in the repo root. Each morning, paste that day's section as the task. Each evening, run the day's review checklist and add an entry to `PROGRESS.md` (done, key numbers, problems, next step). Don't start a day until the previous day's checklist passes.

Dataset facts

Item	Value
Source	Fedotov et al. 2025, public Google Drive folder
Crop	Spring wheat, cultivar Saratovskaya 74 (highly susceptible)
Pots	12 total: 6 inoculated, 6 healthy controls
Camera	Cubert Ultris 20 snapshot, indoor darkroom
Spectral range	450–874 nm, 106 bands, 4 nm steps
Image size	410 × 410 pixels × 106 bands
Time points	Days 4–9 after inoculation (DPI 4–9), 144 images/day (72 healthy, 72 infected)
Total	864 images
Stages	DPI 4–5 pre-symptomatic · DPI 6–7 slight yellowing · DPI 8–9 visible pustules

Benchmarks to compare against

Method	Overall F1	DPI 4 F1	Source
Traditional ML, tuned preprocessing	0.86–0.94	up to 0.83	Fedotov et al. 2025
SimpleCNN, no weighting	0.814	0.937	Zhu et al. 2026
Best model with weighted loss	0.912–0.942	up to 0.99	Zhu et al. 2026

Hard rules (never break these)

1. **Split by pot, never by image.** The same plants are photographed every day, so a random image split leaks near-duplicates and inflates scores. Default split: 8 pots train, 2 validation, 2 test, each balanced healthy/infected.
2. **Report every metric per DPI**, not only overall. DPI 4 F1 is the headline number.
3. **Never tune anything on test pots.** Hyperparameters, thresholds and feature choices use validation pots only.
4. **Fixed seeds.** Every deep learning result = 5 seeds, mean ± standard deviation.
5. **Every experiment writes its config and results** to `results/<experiment>/` as JSON + CSV. No number goes in the report unless it traces to a file.
6. **If pot IDs cannot be recovered from the data, stop and ask.** The whole evaluation depends on them.

Repo structure

```
wheat-rust-early/
  CLAUDE.md            # this section
  PROGRESS.md          # daily log
  requirements.txt
  configs/             # one YAML per experiment
  data/
    raw/               # downloaded cubes (not in git)
    cache/             # masked mean spectra, pooled cubes
    index.csv          # path, dpi, label, pot_id
    splits.json        # train/val/test pots + CV folds
    DATA_CARD.md
  src/
    data/              # loader, masking, caching, splits
    features/          # spectra preprocessing, vegetation indices
```

```
    models/            # classical + deep models
    train.py
    evaluate.py        # per-DPI metrics
    explain.py         # gradients, SHAP, band importance
    predict.py
  notebooks/           # EDA and figures only
  results/
  app/                 # Gradio demo
  tests/
  report/
```

Stack: Python 3.11, NumPy, SciPy, scikit-learn, XGBoost, PyTorch, `spectral` or `h5py` (depending on file format), matplotlib, SHAP, Gradio. Run on Kaggle or Colab GPU if no local GPU.

Day 1 — Setup, download, inspect, explore

Goal: the data is on disk, you know its exact format, every image has a DPI, a label and a pot ID, and you have seen with your own eyes that day-4 infected plants look healthy.

Why this matters: every later result depends on correct labels and pot IDs. The exploration also tells you early whether a spectral signal exists at all.

Concepts to understand today

- A hyperspectral cube is an image with 106 layers instead of 3. Each pixel is a full reflectance curve across 450–874 nm.
- DPI (days post inoculation) is the clock of the experiment. Symptoms appear around DPI 6–7.
- 450–700 nm is visible light (pigments, chlorophyll); 700–874 nm is near-infrared (leaf structure, water).

Tasks for Claude Code

1. Create the repo structure, virtual environment, `requirements.txt`, `CLAUDE.md` and an empty `PROGRESS.md`.
2. Check the Drive folder's total size before downloading. If it is over ~40 GB, download DPI 4, 5 and 8 first and the rest later.
3. Inspect a few files: format (.npy, .mat, .hdr/.raw, .h5 or other), data type, shape (expect 410 × 410 × 106), wavelength list, and whether values are raw counts or reflectance.

4. Work out how DPI, healthy/infected and pot are encoded (folder names, file names or a metadata file). Write it all down in `data/DATA_CARD.md`.
5. Build `data/index.csv` with columns: path, dpi, label, pot_id, height, width, bands.
6. Exploration notebook `notebooks/01_eda.ipynb`:
   a. True-colour images (660 / 550 / 470 nm as red / green / blue), healthy vs infected, at DPI 4, 6 and 8.
   b. Mean spectrum per class per DPI, with a shaded ±1 standard deviation band.
   c. Difference spectrum (infected minus healthy) for each DPI on one plot.

Deliverables: `index.csv`, `DATA_CARD.md`, `01_eda.ipynb`, figures in `results/eda/`.

End-of-day review

- [ ] Counts match: 864 images, 144 per DPI, 72 healthy and 72 infected per DPI
- [ ] Every image has a pot ID, and there are exactly 12 unique pots
- [ ] DPI 4 true-colour images of healthy and infected plants look alike to you
- [ ] The difference spectrum is visibly non-zero by DPI 6 (if it is flat everywhere, check the labels before going further)
- [ ] `PROGRESS.md` entry written

Day 2 — Data pipeline and leak-free splits

Goal: one function returns clean, plant-only, cached data with pot-level splits that are tested for leakage.

Why this matters: raw cubes are slow to load and include pot, soil and background pixels. A bad split is the most common way projects like this report fake 99% results.

Concepts to understand today

- **Leakage:** if pot 3's day-4 image is in training and its day-5 image is in testing, the model can recognise the pot rather than the disease.
- **NDVI masking:** NDVI = (R800 − R650) / (R800 + R650). Healthy leaves reflect strongly at 800 nm and absorb at 650 nm, so a threshold (0.3 in the paper) separates plants from background.

Tasks for Claude Code

1. Loader: read one cube into a float32 array of shape (H, W, 106). If the data card shows raw counts, apply the calibration described in the Fedotov paper.

2. NDVI mask with threshold 0.3. Save 10 mask previews (true-colour image + mask overlay) to check it covers leaves only.
3. Build two caches:
   a. Mean plant spectrum per image (106 values), plus its standard deviation → `data/cache/spectra.npz`.
   b. 4× average-pooled cube (about 103 × 103 × 106) per image → `.npy` files or one Zarr/HDF5 store.
4. Pot-level split: 8 train pots, 2 validation, 2 test, each split holding equal inoculated and control pots. Save to `data/splits.json`.
5. Also generate 6 leave-two-pots-out cross-validation folds (each test fold = 1 inoculated + 1 control pot) for Day 7. Save in the same file.
6. Tests in `tests/test_data.py`: no pot appears in two splits; healthy/infected balance per DPI per split; shapes correct; no NaN or infinite values.

Deliverables: `src/data/`, cache files, `splits.json`, mask previews, passing tests.

End-of-day review

- [ ] `pytest` passes
- [ ] Mask previews cover leaves, not the pot rim or soil
- [ ] Loading the full cached dataset takes under a minute
- [ ] You can explain in one sentence why a random image split would leak
- [ ] `PROGRESS.md` entry written

Day 3 — Traditional ML baseline

Goal: an honest classical baseline on mean plant spectra, with F1 reported for every DPI. Target: overall F1 near 0.86–0.94 and DPI 4 near 0.83, the range Fedotov et al. reported.

Why this matters: a baseline tells you whether deep learning is worth its cost. Fedotov et al. showed preprocessing alone moved F1 from 0.67–0.75 to 0.86–0.94.

Concepts to understand today

- **Savitzky–Golay smoothing** removes sensor noise from a spectrum without flattening its peaks.
- **Normalisation (SNV)** removes brightness differences between images, so the model compares curve shape, not overall light level.

- **First derivative** turns the curve into its slope. It removes baseline offsets and highlights where reflectance changes fastest, such as the red edge near 700 nm.

Tasks for Claude Code

1. Preprocessing variants on mean spectra: raw; smoothed; smoothed + normalised; smoothed + normalised + first derivative.
2. Models: logistic regression, SVM (RBF kernel), Random Forest, XGBoost, PLS-DA.
3. Tune hyperparameters on validation pots only, with a small grid. Evaluate once on test pots.
4. Two training regimes:
   a. Train on all DPIs, evaluate per DPI.
   b. Train on DPI 4–5 only (the pure early-detection task), evaluate on DPI 4–5.
5. Save a results table (model × preprocessing × DPI) and a per-DPI F1 line chart to `results/classical/`.

Deliverables: `src/models/classical.py`, `src/evaluate.py` (reused all project), results table and chart.

End-of-day review

- [ ] Per-DPI F1 exists for every model × preprocessing combination
- [ ] Best result compared with the benchmark. Far below → revisit preprocessing. Near 1.0 at DPI 4 → suspect leakage and re-check splits.
- [ ] Confusion matrix at DPI 4 saved
- [ ] You can explain why the derivative helps classical models
- [ ] `PROGRESS.md` entry written

Day 4 — Vegetation indices and band selection

Goal: find the smallest set of wavelengths that still detects day-4 infection, and show F1 as a function of the number of bands.

Why this matters: a hyperspectral camera costs far more than a 3–5 band multispectral camera. If 3–5 bands work, a farm-ready device becomes realistic. This is your bridge from lab result to practical impact.

Concepts to understand today

- A **vegetation index** combines 2–4 bands into one number, such as NDVI. Each index targets something physical: chlorophyll, anthocyanin, water or structure.
- **Exhaustive band search** tries every pair of bands as a normalised difference (b1 − b2) / (b1 + b2), about 5,565 pairs for 106 bands.
- Zhu et al. found 530.8 nm in the best 2-band and 3-band combinations. Their best 2-band index (MGRVI, 530.8 and 542.9 nm) reached F1 0.900.

Tasks for Claude Code

1. Compute standard indices on mean spectra: NDVI, GNDVI, PRI, ARI/ARI2, MGRVI, GARI, red-edge indices. Use the nearest available band for each wavelength.
2. Exhaustive 2-band normalised-difference search on training pots. Rank by validation F1 on DPI 4–5.
3. Permutation importance from the best Random Forest / XGBoost, then forward feature selection to k = 1, 2, 3, 5, 10 bands.
4. Plot band importance across wavelength, shading 480–550 nm and 750–870 nm.
5. Plot DPI 4 F1 against number of bands (1, 2, 3, 5, 10, 106).

Deliverables: `src/features/indices.py`, `results/bands/` with the ranked index table and both plots.

End-of-day review

- [ ] Best 2-band index and its DPI 4 F1 logged
- [ ] F1-vs-number-of-bands curve saved
- [ ] Your top bands compared with 480–550 nm. Agreement with the paper is evidence the signal is real.
- [ ] Top 5 bands saved to a file for the Day 8 sensor experiment
- [ ] `PROGRESS.md` entry written

Day 5 — Deep learning I: training harness, SimpleCNN, 1D-CNN

Goal: a reusable training harness and the two simplest deep models, with per-DPI F1 over 5 seeds. Benchmark: SimpleCNN reached overall F1 0.814 and DPI 4 F1 0.937 (±0.13) in Zhu et al.

Why this matters: with only 864 images, Zhu et al. found the simplest network beat complex ones. Start simple and earn every bit of complexity.

Concepts to understand today

- **SimpleCNN** treats the 106 bands as input channels to ordinary 2D convolutions, the same way an RGB network treats 3 channels.
- **1D-CNN** slides filters along the spectrum of one pixel or one mean curve. It learns spectral shape only, with no spatial information.
- **Spectrum-safe augmentation:** flips, 90° rotations and small random crops are fine. Colour jitter, brightness changes or hue shifts are not: they alter the very signal you are trying to detect.

Tasks for Claude Code

1. PyTorch `Dataset` over the cached pooled cubes, with MinMax normalisation to [0, 1] fitted on training pots only.
2. Training loop from a YAML config: Adam, learning rate 1e-4, weight decay 1e-5, batch size 4, up to 50 epochs, early stopping with patience 10 on validation loss. Log every epoch to CSV.
3. Models: SimpleCNN (pooled cubes) and 1D-CNN (mean spectra, plus a variant on randomly sampled plant pixels).
4. Run 5 seeds (42, 123, 456, 789, 1024). Report per-DPI F1 as mean ± standard deviation.
5. **Sanity check:** retrain SimpleCNN with shuffled labels. F1 should fall to about 0.5. If it stays high, something is leaking.

Deliverables: `src/train.py`, `src/models/simple_cnn.py`, `src/models/cnn1d.py`, `results/dl_v1/`.

End-of-day review

- [ ] Training and validation curves look sane (no instant 100% accuracy, no exploding loss)
- [ ] Per-DPI table with mean ± std for both models
- [ ] Shuffled-label run lands near F1 0.5
- [ ] Compared with the Day 3 classical baseline at DPI 4
- [ ] `PROGRESS.md` entry written

Day 6 — Deep learning II: stronger models, weighted loss, ablations

Goal: test two spectral-specific architectures and the weighted-loss trick, then pick one final model with a written reason. Benchmark: weighting raised overall F1 to 0.912–0.942, and HybridSN's DPI 4 F1 went from 0.68 to 0.93.

Why this matters: the weighted loss was the single biggest gain in Zhu et al. Ablations tell you which steps actually help instead of guessing.

Concepts to understand today

- **HybridSN** uses 3D convolutions first (learning patterns across neighbouring bands) and then 2D convolutions (learning spatial patterns).
- **SpectralAttentionNet** learns a weight per band, like a Squeeze-and-Excitation block, so the model decides which wavelengths to trust.
- **DPI-weighted loss** multiplies the loss on hard samples (here, hard DPIs) by 1.2–1.5, so training spends more effort on them.
- **Ablation** = remove one step at a time and measure what it was worth.

Tasks for Claude Code

1. Implement HybridSN and SpectralAttentionNet in the Day 5 harness.
2. From the Day 5 per-DPI results, identify the weakest DPIs (the paper's were DPI 7–8). Apply loss weights W = 1.0, 1.2, 1.3, 1.5 to those DPIs.
3. Ablations on the best model: raw input vs NDVI mask vs NDVI mask + first derivative. (In the paper, the mask added +2.6% F1 and the derivative cost −4.7% for deep models.)
4. Optional: train on DPI 4–5 + matching healthy images only.
5. Master results table: model × setting × DPI, all with mean ± std over 5 seeds.

Deliverables: `src/models/hybridsn.py`, `src/models/spectral_attention.py`, `results/dl_v2/`, master table.

End-of-day review

- [ ] Master table complete
- [ ] Final model chosen, with a short written reason: DPI 4 F1, standard deviation across seeds, model size
- [ ] Any per-DPI drop caused by weighting noted (the paper saw local drops at DPI 6 and DPI 7 for some models)
- [ ] `PROGRESS.md` entry written

Day 7 — Robustness, cross-validation, lead time

Goal: prove the results are not luck of one split, and state how many days before visible symptoms the model detects infection, with a confidence interval.

Why this matters: the published test set is only 2 pots, so one unlucky pot swings results a lot (Zhu et al.'s DPI 4 standard deviations reached ±0.13 to ±0.37). Cross-validation over all pots is the step that makes your numbers trustworthy, and most student projects skip it.

Concepts to understand today

- **Leave-pot-out cross-validation:** every pot takes a turn in the test set, so the final score uses all 12 pots.
- **Lead time:** the earliest DPI at which the model flags an infected pot, compared with the symptom day (DPI 6).
- **Calibration:** whether a predicted 80% really means infected about 80% of the time. It matters when choosing an alert threshold.

Tasks for Claude Code

1. Run the 6 leave-two-pots-out folds from `splits.json` for the best classical model and the best deep model. Hyperparameters stay fixed from earlier days.
2. Bootstrap 95% confidence intervals for DPI 4 and DPI 5 F1.
3. Pick an alert threshold on validation pots (for example, the one giving 95% specificity on healthy plants). Then for each inoculated test pot, record the first DPI where its images pass the threshold.
4. ROC and precision-recall curves per DPI; a reliability (calibration) plot.
5. Paired comparison of classical vs deep across folds (Wilcoxon signed-rank test).

Deliverables: `results/cv/` with fold-level and summary tables, lead-time table, curves.

End-of-day review

- [ ] Cross-validated mean ± std reported. A big gap from the single-split result means the single split was lucky, and the report must say so.
- [ ] Lead time stated in days, with its interval
- [ ] False-alarm rate on healthy pots at the chosen threshold stated
- [ ] Classical vs deep difference stated as significant or not
- [ ] `PROGRESS.md` entry written

Day 8 — Interpretability and the RGB vs hyperspectral experiment

Goal: show which wavelengths drive early detection, and prove whether a normal RGB camera could do the same job.

Why this matters: the RGB-only experiment answers the first question any examiner will ask: "why not just use a phone camera?" It also turns your band analysis into a concrete sensor recommendation.

Concepts to understand today

- **Input gradients** show how sensitive the prediction is to each band (local sensitivity).
- **SHAP** estimates how much each band contributes on average (global contribution). The two can disagree, and that disagreement is informative.
- In Zhu et al., gradients pointed to 480–550 nm (chlorophyll-related, key for pre-symptomatic detection) while SHAP pointed to 750–870 nm (leaf structure, general disease presence).

Tasks for Claude Code

1. Input-gradient band importance for the final deep model, overall and per DPI.
2. SHAP (GradientExplainer, 50 training images as background) for 2D models. Skip 3D models if memory runs out, as the paper did.
3. Sensor ablation: retrain the final model on (a) RGB only (470 / 550 / 660 nm), (b) the top 5 bands from Day 4, (c) all 106 bands. Report DPI 4 and DPI 5 F1 for each, with 5 seeds.
4. Per-pixel infection maps: apply the 1D model per plant pixel and overlay the probabilities on the true-colour image for a few DPI 4 and DPI 8 samples.
5. One combined figure: gradient importance, SHAP importance and Day 4 index bands on the same wavelength axis.

Deliverables: `src/explain.py`, `results/explain/`, sensor ablation table, infection map figures.

End-of-day review

- [ ] RGB-only DPI 4 F1 recorded. If it is close to 0.5, that is your strongest argument for spectral sensing.
- [ ] Top-5-band DPI 4 F1 recorded: this is the multispectral sensor case
- [ ] Gradient, SHAP and index results compared, and agreements or conflicts written down
- [ ] `PROGRESS.md` entry written

Day 9 — Inference package and demo

Goal: one command runs a demo that shows a plant that looks healthy next to the model's "infected" verdict and its infection map.

Why this matters: that contrast (looks healthy, model says infected) is the whole project in one picture. It is what reviewers and your FYP panel will remember.

Tasks for Claude Code

1. `src/predict.py`: input a cube path → infected probability, verdict at the Day 7 threshold, per-pixel infection map.
2. Export the final model (TorchScript or ONNX) and measure CPU inference time per cube.
3. Gradio app in `app/`:
   a. Pick a test sample (DPI 4–9, healthy or infected).
   b. Left: true-colour image. Middle: infection heatmap. Right: verdict, probability, true label, and "symptoms expected around day 6–7" for early samples.
   c. A second tab: the band-importance figure and the RGB vs hyperspectral table from Day 8.
4. Model card `MODEL_CARD.md`: data, intended use, measured performance, limits (one cultivar, indoor lab, 12 pots).
5. Optional, if time allows: a short note on how this would combine with a weather-based rust risk layer for Punjab wheat. Note only, no build.

Deliverables: `predict.py`, exported model, working `app/`, `MODEL_CARD.md`.

End-of-day review

- [ ] App runs from a clean environment with one command
- [ ] Predictions in the app match the evaluation outputs for the same samples
- [ ] At least one DPI 4 sample correctly flagged as infected, ready for screenshots
- [ ] `PROGRESS.md` entry written

Day 10 — Report, figures, final review

Goal: a reproducible repo and a report where every number traces to a results file, plus slides and viva answers.

Why this matters: a strong result with sloppy reporting reads as a weak one. The fresh-eyes audit catches leakage and over-claims before an examiner does.

Tasks for Claude Code

1. `make figures` (or one script) regenerates every report figure from `results/`.

2. Report, 8–12 pages:
    a. Introduction: why pre-symptomatic detection matters; stem rust threat; the DPI 6–7 symptom window.
    b. Related work: Fedotov et al. 2025, Zhu et al. 2026, tomato bacterial spot (Zhang et al. 2024), PSNet low-cost camera.
    c. Data and pot-level evaluation protocol.
    d. Methods: classical pipeline, deep models, weighted loss.
    e. Results: per-DPI tables, cross-validation, lead time, false-alarm rate.
    f. Interpretability and the RGB vs multispectral vs hyperspectral comparison.
    g. Limitations: one cultivar, indoor darkroom, controlled inoculation, 12 pots.
    h. Future work: field data, a 3–5 band multispectral sensor, fusion with weather risk for Pakistani wheat.
3. `README.md` with exact steps to reproduce from a fresh clone.
4. **Fresh-eyes audit:** open a new Claude Code session with no prior context. Ask it to check for leakage, for claims not backed by a results file, and for numbers that differ between report and results.
5. 8–10 slides for a 5-minute talk, plus a list of 10 likely viva questions with answers.

Deliverables: `report/`, slides, `README.md`, audit notes and fixes.

End-of-day review

- [ ] Every number in the report traces to a file in `results/`
- [ ] A fresh clone reproduces the main table
- [ ] Audit findings fixed or written up as limitations
- [ ] Limitations stated plainly, not hidden
- [ ] Final `PROGRESS.md` entry written

Likely viva questions to prepare: Why split by pot? Why not RGB? Is DPI 4 really symptom-free? How would this work in a field? What is your false-alarm rate? Why did the simple model win? How many bands does a real sensor need?

Risks, cut list, references

Risks and fallbacks

Risk	Fallback
Dataset is too large for Colab/Kaggle disk (rough estimate 30–60 GB, unconfirmed)	Download DPI 4, 5 and 8 first; cache pooled cubes and delete raw files after caching
Pot IDs are not recoverable from file names or metadata	Stop on Day 1; check the Fedotov paper's data description or contact the authors before building anything
Files are raw counts, not reflectance	Apply the calibration in the Fedotov paper; record it in `DATA_CARD.md`
Results swing a lot between seeds or splits	Expected with 2 test pots; rely on Day 7 cross-validation and report intervals
GPU time runs out	Use mean-spectra models (classical + 1D-CNN) as the main result; they run on CPU
Running behind schedule	Use the cut list below

Cut list (drop in this order if behind)

1. Day 9 optional weather note
2. SpectralAttentionNet (keep SimpleCNN + HybridSN)
3. Per-pixel infection maps (Day 8, task 4)
4. Gradio app second tab
5. Calibration plots (Day 7, task 4)

Never cut: the pot-level split, per-DPI metrics, cross-validation, the RGB vs hyperspectral experiment, and band importance. Those five make the project credible.

References

- Fedotov et al. 2025 — Advanced hyperspectral image processing and ML for early detection of wheat stem rust
- Zhu et al. 2026 — Pre-symptomatic detection of wheat stem rust using hyperspectral imaging and deep learning

- Dataset (Google Drive)
- Terentev et al. 2025 — Cross-crop transferability for early stem rust detection in wheat and barley
- Zhang et al. 2024 — Hyperspectral imaging for early detection of tomato bacterial leaf spot
- PSNet 2026 — Low-cost hyperspectral + RGB fusion for presymptomatic detection